



南開大學
Nankai University

密码与网络空间安全学院
深度学习及应用（高阶课）实验报告

循环神经网络网络算法

姓名：陈培杰
学号：2312991
专业：信息安全

2026 年 5 月 24 日

目录

1 网络结构	1
2 关键代码设计	1
3 实验结果	3
3.1 训练曲线	3
3.2 测试集结果与分析	4
3.3 矩阵预测图	5
3.4 分类报告与评价指标分析	6
4 解释为什么 LSTM 网络的性能优于 RNN 网络	6
4.1 实验现象的回顾与印证	7
4.2 基础 RNN 梯度消失的数学根源	7
4.3 LSTM 门控机制与加法梯度的数学解法	7
5 选做：姓名生成（从国籍到姓名）	8
5.1 训练过程与隐状态聚类分析	8
5.2 测试评估与生成结果	8

1 网络结构

老师提供的 RNN 网络结构如下：

```
1 CharRNN(  
2     (rnn): RNN(56, 128)  
3     (h2o): Linear(in_features=128, out_features=18, bias=True)  
4 )
```

网络结构较为简单，构建一层 RNN，然后取最后一个时间步的 hidden 作为全连接层的输入，通过全连接层进行概率预测。

2 关键代码设计

LSTM_Cell 的代码如下。

```
1 class LSTMCell(nn.Module):  
2     def __init__(self, input_size, hidden_size):  
3         super().__init__()  
4         self.input_size = input_size  
5         self.hidden_size = hidden_size  
6         weight_scale = 0.01  
7         self.w_f = nn.Parameter(torch.randn(hidden_size, input_size + hidden_size) *  
8             weight_scale)  
9         self.w_i = nn.Parameter(torch.randn(hidden_size, input_size + hidden_size) *  
10             weight_scale)
```

```

9         self.w_c = nn.Parameter(torch.randn(hidden_size, input_size + hidden_size) *
10                                   weight_scale)
11         self.w_o = nn.Parameter(torch.randn(hidden_size, input_size + hidden_size) *
12                                   weight_scale)
13         self.b_f = nn.Parameter(torch.zeros(hidden_size))
14         self.b_i = nn.Parameter(torch.zeros(hidden_size))
15         self.b_c = nn.Parameter(torch.zeros(hidden_size))
16         self.b_o = nn.Parameter(torch.zeros(hidden_size))
17     def forward(self, input, h_t_1, c_t_1):
18         if input.dim() == 1:
19             input = input.unsqueeze(0)
20         if h_t_1.dim() == 1:
21             h_t_1 = h_t_1.unsqueeze(0)
22         if c_t_1.dim() == 1:
23             c_t_1 = c_t_1.unsqueeze(0)
24         cat_input = torch.cat([h_t_1, input], dim=-1)
25         f_t = torch.sigmoid(cat_input @ self.w_f.t() + self.b_f)
26         i_t = torch.sigmoid(cat_input @ self.w_i.t() + self.b_i)
27         candidate_c = torch.tanh(cat_input @ self.w_c.t() + self.b_c)
28         o_t = torch.sigmoid(cat_input @ self.w_o.t() + self.b_o)
29         c_t = f_t * c_t_1 + i_t * candidate_c
30         h_t = o_t * torch.tanh(c_t)
31         return h_t, c_t

```

LSTM 主干中 forward 的主要逻辑代码如下。这里我设置了 lengths 参数, lengths 是一个张量, 维度为 [batch_size], 用于构造 mask。由于老师提供的训练代码是写一个 for 循环, 一个一个从 dataset 中取数据, 这样太慢了; 所以我想用 dataloader, 用 dataloader 得对原数据进行 pad 并设置 mask。

```

1 def forward(self, X, lengths=None):
2     ... ..
3     if lengths is not None:
4         mask = torch.arange(seq_len, device=X.device).unsqueeze(1) <
5             lengths.to(X.device).unsqueeze(0)
6     else:
7         mask = None
8     for t, x, in enumerate(X):
9         layer_input = x
10        for layer in range(self.num_layers):
11            lstm_cell = self.cells[layer]
12            new_h, new_c = lstm_cell(layer_input, h_states[layer], c_states[layer])
13
14            if mask is not None:
15                step_mask = mask[t].unsqueeze(1)
16                h_states[layer] = torch.where(step_mask, new_h, h_states[layer])
17                c_states[layer] = torch.where(step_mask, new_c, c_states[layer])
18            else:
19                h_states[layer] = new_h
20                c_states[layer] = new_c

```

```
layer_input = h_states[layer]
```

GRU 的代码和 LSTM 的代码差别不大，将注册的参数和计算的逻辑换一下就可以了。

3 实验结果

模型命名规则举例：RNN_1 表示 1 层 RNN，RNN_2 表示 2 层 RNN。LSTM 和 GRU 同理。

3.1 训练曲线

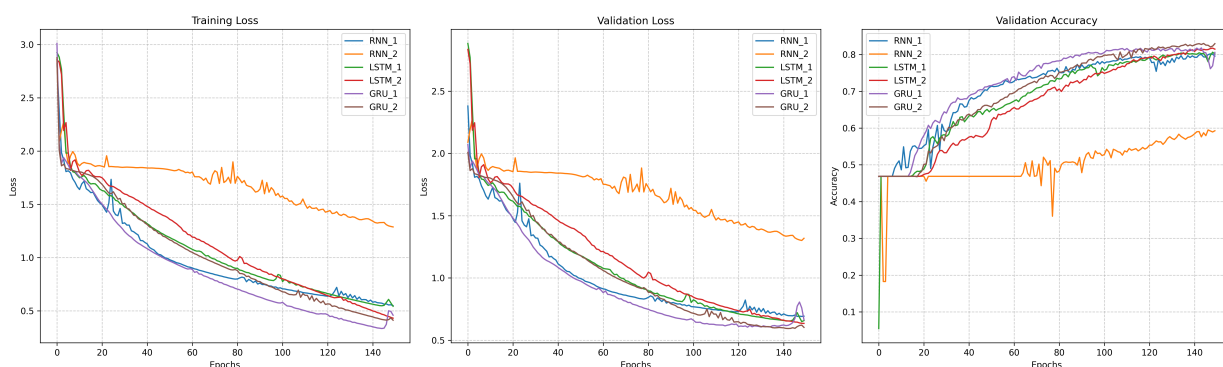


图 3.1: learning_curves_comparison

从图 3.1 的训练损失 (Training Loss)、验证损失 (Validation Loss) 和验证准确率 (Validation Accuracy) 曲线中，我们可以得出以下关于模型架构与网络深度的关键结论：

- 不同循环神经网络架构的性能对比：**整体而言，带有门控机制的 GRU 网络在本次名字字符级序列预测任务中表现最为出色。特别地，单层 GRU (GRU_1) 在三个指标上均取得了最佳表现：其训练损失和验证损失下降最快且收敛值最低，最终验证准确率也最高（超过 80%）。虽然 LSTM 和 GRU 都能通过门控机制有效捕捉序列中的依赖关系，但 GRU 参数量相对较少，在名字识别这种相对中等复杂度的字符级建模任务中，优化效率更高，展现出了更好的泛化性能。基础的 RNN_1 虽然初期收敛较快，但在后期的准确率提升上略逊于 GRU。
- 网络深度（层数）对训练动态的影响：**在本实验中，增加网络层数虽然未能带来最终预测性能的显著提升（单层网络的最终指标普遍优于或等同于双层网络），但显著改变了模型的训练动态过程。具体表现如下：
 - 深层 RNN 的退化现象：**图中最显著的异常在于双层普通循环神经网络 (RNN_2, 橙色曲线)。其损失值居高不下且波动剧烈，验证准确率在前 60 个 Epoch 停滞在 0.47 左右，最终仅勉强达到约 0.60。这直观地反映了基础 RNN 缺乏门控机制的致命弱点：随着网络深度的增加以及序列在时间步上的反向传播，极易引发严重的梯度消失问题，导致深层权重无法得到有效更新。
 - 深层 LSTM 与 GRU 的前期收敛阻力：**对于 LSTM_2 和 GRU_2，得益于细胞状态或门控机制的引入，它们成功克服了深层 RNN 的梯度消失问题，最终能够收敛到与单层模型相近的水平（约 80% 的准确率）。然而，相比于单层模型，双层模型在前期的收敛速度稍慢（例如 LSTM_2 的初期准确率爬升明显慢于 LSTM_1）。

- **深层网络训练过程更加稳定：**尽管双层模型前期收敛较慢，但观察曲线细节可以发现，带有门控机制的双层网络（LSTM_2 与 GRU_2）在整个训练过程中的各项指标均表现得更为稳定。具体而言，在训练/验证损失下降以及准确率上升的过程中，双层网络曲线的局部波动明显少于单层网络。特别是在训练末期（高 Epoch 阶段），单层网络（LSTM_1 和 GRU_1）的损失和准确率曲线均出现了轻微的振荡与反弹迹象，而相应的双层网络曲线则始终保持着极佳的平滑度。这表明较深的网络结构在参数空间中可能找到了一处更为平缓的局部最优解，在训练后期展现出了更强的抗扰动能力与稳定性。
3. 过拟合风险分析：观察各模型的训练和验证损失曲线可以发现，除了 RNN_2 表现异常外，其余模型的验证损失与训练损失同步下降，且在 150 个 Epoch 内验证曲线并未出现明显的“U 型”反弹上升趋势。这说明模型在训练过程中未出现严重的过拟合现象，当前的训练轮数和模型容量匹配度较好。

3.2 测试集结果与分析

为了进一步评估各模型的泛化能力，我们在独立的测试集上进行了性能评估。各模型在测试集上的准确率（Test Accuracy）如表 1 所示。

表 1: 不同模型在测试集上的预测准确率对比

模型架构 (Model)	测试准确率 (Test Accuracy)
RNN_1 (单层 RNN)	80.88%
RNN_2 (双层 RNN)	79.58%
LSTM_1 (单层 LSTM)	81.18%
LSTM_2 (双层 LSTM)	80.53%
GRU_1 (单层 GRU)	82.12%
GRU_2 (双层 GRU)	81.57%

通过对表 1 中的数据进行对比，可以得出以下结论：

1. **模型架构性能排序：**在相同层数的情况下，不同架构的性能表现呈现出 **GRU > LSTM > RNN** 的趋势。其中，单层 GRU（GRU_1）达到了最高的测试准确率（82.12%）。这再次验证了在字符级序列建模中，GRU 通过重置门和更新门能更高效地捕捉长距离依赖信息，且由于参数量相对 LSTM 较少，在当前规模的数据集上更易于收敛到较优的参数空间。
2. **层数对泛化能力的影响：**值得关注的是，所有架构均表现出 **单层网络优于双层网络** 的特征（如 RNN_1 比 RNN_2 高出 1.30%）。不过这也带有一定的**随机性**，后续跑了几次，也有 2 层准确率大于 1 层的。
深层训练难度：增加第二层网络虽然提升了理论上的模型容量，但在反向传播过程中，即便是带有门控的 LSTM/GRU，在堆叠多层后依然会增加梯度流动的复杂性。结合测试集表现略有下降的事实，推测双层模型在当前数据集规模下可能出现了轻微的过拟合，或者在 150 个 Epoch 内未能完全优化至其理论上限。
3. **基础 RNN 的局限性：**RNN_2 的测试准确率为所有模型中最低（79.58%），这与其在训练过程中表现出的梯度消失问题直接相关。由于基础 RNN 无法有效维持远距离的状态信息，其对长姓名的识别能力显著弱于门控神经网络。

3.3 矩阵预测图

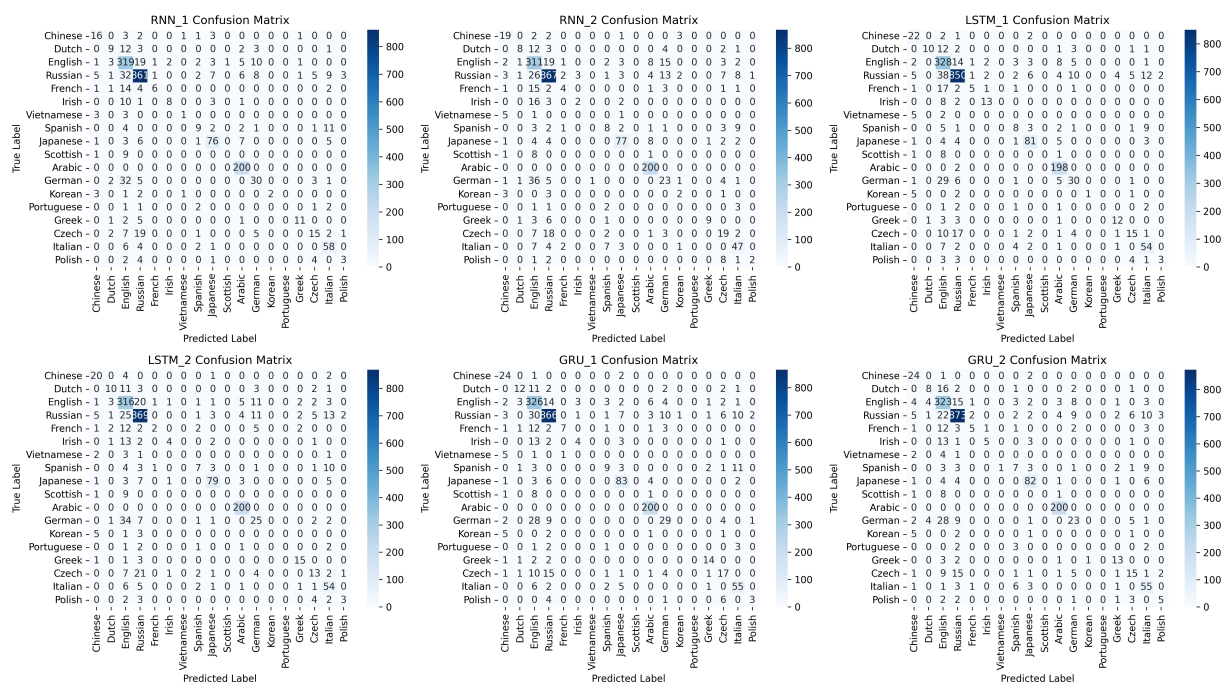


图 3.2: confusion_matrices_grid

图 3.2 展示了六个不同结构的模型在测试集上的混淆矩阵。通过对混淆矩阵的详细观察，我们可以更直观地分析模型在各个具体国籍类别上的预测偏好与分类瓶颈：

- 数据分布与类别不平衡 (Class Imbalance):** 从矩阵对角线以及整体的颜色深浅可以明显看出，该数据集存在严重的类别不平衡（长尾分布）问题。**English** 和 **Russian** 类别的样本数量远超其他类别（对应最深蓝色的色块）。这种数据倾斜导致所有模型都表现出了一定的“多数类偏好”，即当模型对某些少数类样本（如 **German**、**French**、**Czech** 等）的特征把握不准时，极易将其错误预测为数据量最大的 **English** 或 **Russian**（表现为这两列非对角线上的数值偏高）。
- 语系关联与字符特征重叠:** 混淆矩阵直观地反映了不同国籍名字在字符层面的相似度与区分度。
 - 易混淆群体:** 欧洲国家的姓名（如 **English**、**French**、**German**、**Dutch**）由于多基于拉丁字母，且由于地缘和历史因素共享许多相似的字符组合（如特定的前缀、后缀），在矩阵中表现出较高的相互误判率。例如，大量的 **German** 和 **Dutch** 名字被模型错分为 **English**。
 - 高区分度群体:** 相对而言，**Arabic**、**Japanese** 和 **Chinese** 等类别的拼音或罗马音转写具有非常独特的字符序列模式（例如日语罗马音中规律的元辅音交替）。因此，尽管这些类别的样本量远不及英俄两国，但矩阵显示模型对它们的预测非常集中在对角线上，识别准确率高，较少与其他类别发生混淆。
- 不同模型的分​​类细节差异:** 对比这六个矩阵，可以进一步印证前文的结论。性能较差的 **RNN_2** 在非对角线上的“噪点”分布更为分散，尤其是在将少数类别误判为 **English** 的区域，其误报数值明显高于其他模型，说明深层 RNN 提取区分性特征的能力较弱。而表现最佳的 **GRU_1** 在对角线上的能量分布更为集中，它在一定程度上改善了对中等规模类别（如 **Arabic**、**Italian**）的识别能力，有效降低了因数据不平衡带来的假阳性错误。

3.4 分类报告与评价指标分析

为了更全面地评估模型在不平衡数据集上的综合表现，我们提取了各模型在测试集上的核心评价指标（包含准确率、宏平均与加权平均），整理如表 2 所示。

表 2: 各模型在测试集上的总体评价指标对比

模型架构	准确率 (Accuracy)	宏精确率 (Macro P)	宏召回率 (Macro R)	宏 F1 分数 (Macro F1)	加权 F1 分数 (Weighted F1)
RNN_1	80.88%	0.58	0.44	0.48	0.79
RNN_2	79.58%	0.49	0.40	0.42	0.78
LSTM_1	81.18%	0.58	0.45	0.47	0.80
LSTM_2	80.53%	0.48	0.41	0.42	0.79
GRU_1	82.12%	0.51	0.45	0.46	0.81
GRU_2	81.57%	0.48	0.43	0.43	0.80

通过对表 2 及其背后具体类别数据的深入剖析，我们可以得出以下关键结论：

- 评价指标的巨大“鸿沟”与类别不平衡的量化印证：**在所有六个模型中，加权 F1 分数均保持在较高水平（最高达 0.81），而宏 F1 分数却出现断崖式下跌（仅在 0.42 - 0.48 之间）。这两项指标之间高达 0.35 左右的巨大差异，是对数据集中长尾分布现象的严谨量化印证。加权指标被数据量庞大的多数类（如 Russian 和 English，F1 分数均在 0.90 左右）主导，掩盖了模型在极少数类（如 Vietnamese、Scottish 等类别上各项指标几乎为 0）上的糟糕表现；而宏平均指标平等对待所有类别，暴露了模型在类别均衡学习能力上的短板。
- 网络深度对少数类别识别的负面影响：**观察表格中的宏 F1 分数可以发现一个非常有趣的现象：**所有双层网络的宏指标均显著低于对应的单层网络**（RNN_2 比 RNN_1 下降 0.06，LSTM_2 比 LSTM_1 下降 0.05，GRU_2 比 GRU_1 下降 0.03）。这表明，在缺乏足够数据支撑的少数类样本上，深层网络由于参数量更大，更容易对多数类产生“过拟合”，从而进一步挤压了对极少数类的预测概率空间，导致其对长尾类别的召回能力进一步恶化。
- 浅层网络与门控机制在特征捕获上的取舍：**对比 RNN_1 和 GRU_1 可以发现一个反直觉的现象：尽管 GRU_1 的总体准确率（0.82）和加权 F1（0.81）均为全局最优，但 RNN_1 却取得了最高的宏 F1 分数（0.48）。结合具体分类报告的明细来看，这是由于 RNN_1 特征提取能力相对较弱，其输出的预测分布包含了更多的“噪声”（表现为非对角线上的误判更分散），这种发散性使得它偶然“蒙中”了少数如 Korean 或 Polish 的样本。相反，GRU_1 的门控机制能够高度集中地拟合区分度明显的特征，它在事实上“放弃”了对特征极度匮乏的极小类别的预测，将置信度集中分配给了多数类和中等类别（如 Arabic 召回率达 1.00），从而实现了整体预测期望的最大化。

4 解释为什么 LSTM 网络的性能优于 RNN 网络

通过前文的实验结果分析（特别是图 3.1 的训练曲线与表 1 的测试准确率），我们可以明确得出 LSTM 网络在“名字到国籍”序列分类任务中的综合性能显著优于基础 RNN 网络。本节将结合本次实验的具体现象，从沿时间反向传播（BPTT）的数学原理出发，深入探讨这一现象的本质原因。

4.1 实验现象的回顾与印证

在我们的实验中，LSTM 与 RNN 的性能差异在网络加深时表现得尤为剧烈。如前文所述，双层基础循环神经网络 (RNN_2) 在训练时遭遇了严重的退化，其验证准确率长期停滞在 0.47 左右，最终测试准确率仅为 79.58%，且损失曲线波动剧烈。相反，双层 LSTM (LSTM_2) 则表现出了极强的抗扰能力，不仅顺利收敛，而且训练过程极其平滑。这一实验现象并非偶然，而是由基础 RNN 固有的数学缺陷所导致的，即在处理长序列时极易发生梯度消失或梯度爆炸问题 [1]。

4.2 基础 RNN 梯度消失的数学根源

在基础 RNN 中，隐藏层状态 h_t 的更新公式为：

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b) \quad (1)$$

在训练过程中，模型通过沿时间反向传播算法更新权重。假设我们要计算 t 时刻的损失 L_t 对 k 时刻 ($k \ll t$) 隐藏层状态 h_k 的偏导数，根据链式法则，我们需要计算一系列雅可比矩阵的乘积：

$$\frac{\partial L_t}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \quad (2)$$

其中，相邻时间步的偏导数为：

$$\frac{\partial h_i}{\partial h_{i-1}} = \text{diag}(\tanh'(W_{hh}h_{i-1} + \dots))W_{hh} \quad (3)$$

Pascanu 等人 [3] 证明了，如果权重矩阵 W_{hh} 的最大奇异值小于 1，或者激活函数（如 $\tanh$ ）的导数（最大值为 1）在大部分区域远小于 1，那么上述连乘项 $\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$ 将随着时间间隔 $(t-k)$ 的增大呈指数级衰减，迅速趋近于 0。在我们的“名字识别国籍”任务中，许多名字（如俄语或阿拉伯语转写）字符较长，当网络试图将句首字符的特征传递到句尾进行分类时，基础 RNN 的梯度已经在反向传播中“消失”殆尽，导致早期的权重无法得到有效更新。这就是 RNN_2 难以训练的理论根源。

4.3 LSTM 门控机制与加法梯度的数学解法

长短期记忆网络 (LSTM) 由 Hochreiter 和 Schmidhuber [2] 提出，其核心在于引入了细胞状态 (Cell State, C_t) 和三个门控结构（遗忘门 f_t 、输入门 i_t 、输出门 o_t ）。LSTM 中细胞状态的更新公式为：

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (4)$$

当我们同样考虑梯度沿时间步反向传播时， $\frac{\partial C_t}{\partial C_{t-1}}$ 的推导结果为：

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t + \dots (\text{其他涉及门控导数的项}) \quad (5)$$

与基础 RNN 致命的“连乘”机制不同，LSTM 细胞状态的梯度传播路径中包含了**加法项**。如果遗忘门 f_t 学习到的值接近于 1（即模型决定“记住”过去的信息），那么 $\frac{\partial C_t}{\partial C_{t-1}} \approx 1$ 。这意味着梯度可以无损地跨越多个时间步向后传递，从而完美地解决了梯度消失问题。

5 选做：姓名生成（从国籍到姓名）

这里选用的模型是 RNN。

5.1 训练过程与隐状态聚类分析

为了监控模型的学习状态并探究其对字符序列本质差异的捕捉能力，我记录了 200 个 Epoch 的损失，并利用 t-SNE 算法对隐藏状态进行了降维可视化。

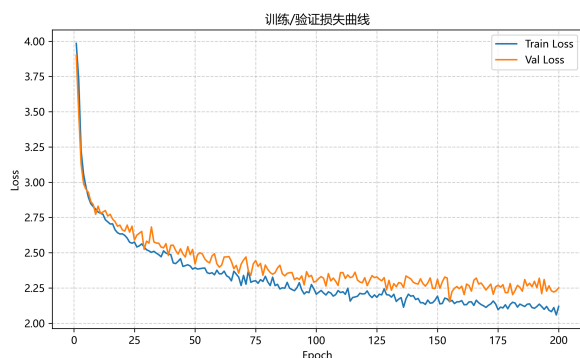


图 5.3: 训练与验证损失曲线

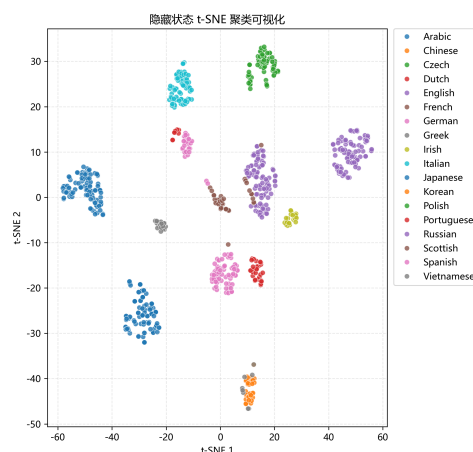


图 5.4: 隐状态 t-SNE 聚类

如图 5.3 所示，模型在前 25 个 Epoch 损失迅速下降，随后平缓收敛至约 2.10（Train）和 2.25（Val）。两曲线走势一致，未出现发散上翘，表明模型具备良好的泛化能力且未发生严重过拟合。同时，图 5.4 显示隐状态在二维空间呈现显著且边界清晰的簇状分布（如 Russian、Chinese 等均形成紧凑聚类）。这证明 RNN 网络不仅学会了记忆字符，更在表征空间成功解耦了不同国家的语言风格特征。但是在英文、法文、德文之间有些重叠。

5.2 测试评估与生成结果

模型在测试集上的平均损失为 **2.250345**，困惑度 (PPL) 为 **9.491014**。较低的困惑度说明模型对目标字符序列的概率预测具有较高的确定性。

基于训练完毕的模型，我们输入特定国籍作为条件进行采样生成，结果及定性分析如表 3 所示。模型极好地捕捉到了各国语言的拼写模式与构词规律。

表 3: 不同国籍生成姓名示例及特征分析

国籍 (条件)	生成的姓名 (采样结果)	定性分析结论
[Russian]	Allovev, Banton, Chanton	成功生成带有典型俄语姓名后缀（如“-ev”）的单词。
[German]	Arter, Banger, Cherr	高度符合印欧语系发音习惯和字符组合概率，具真实感。
[Spanish]	Allan, Bara, Carer	
[Chinese]	Ang, Ban, Chan	汉语拼音书写特征被精确还原，符合中国人名拼音短小的特点。

参考文献

- [1] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.
- [2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [3] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. PMLR, 2013.